Bitcoin and Ethereum Returns Analysis

Summative Assessment Section II

Valdez, Chance Jayden Tan, Heinz Nicole

Preliminaries Before Both Parts

Setup Shared by Both Parts

```
from pathlib import Path
from scipy import stats
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

ROOT = Path.cwd()
while not (ROOT / "data").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

SEED = 51
```

Assumptions Shared by Both Parts

These apply to the BTC analysis in Part 1 and the ETH analysis in Part 2.

1. All timestamps are Unix seconds read as UTC.
2. Returns are log returns of the close price, $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$.
3. The bars with zero volume are removed first, since they are padded rows with a flat price and would create fake zero returns.
4. A return only counts if both bars are exactly one interval apart (1 minute for BTC, 5 minutes for ETH). Missing bars are left as gaps and never filled.
5. We keep data up to the end of September 15, 2026 (UTC), as that is what the assessment requires.
6. We checked that no return is NaN or infinite before running any analysis.

Part 1: Distribution Fitting for BTC/USD (1-Minute Returns)

1.1 Question

Find out which probability distribution function best fits Bitcoin's returns for trading data every minute, from January 1, 2012 to September 15, 2026, for Bitcoin quoted in United States dollars or the BTC/USD pair.

1.2 Data

After the raw bitcoin data has been processed by `resample_btc_data.py`, the processed data with log returns are in `data/processed`. Now let's load it.

```
btc_path = ROOT / "data" / "processed" / "btc_1min_returns_2012_2026.csv"
btc = pd.read_csv(btc_path, index_col=0, parse_dates=True)
r_btc = btc["log_return"].to_numpy() * 100  # Percentage

assert np.isfinite(r_btc).all(), "found NaN or inf in the returns"

print(f"Observations: {len(r_btc):,}")
print(f"First Bar: {btc.index.min()}")
print(f"Last Bar: {btc.index.max()}")
```

```
Observations: 5,983,015
First Bar: 2012-01-03 17:27:00
Last Bar: 2026-09-15 23:59:00
```

BTC Dataset Link: https://www.kaggle.com/datasets/mczielinski/bitcoin-historical-data

1.3 Assumptions

On top of the shared assumptions above:

1. The BTC file starts at 1325376060, which is January 1, 2012 at 00:01 UTC, and runs to September 29, 2026 at 03:09 UTC. We use only the part up to September 15, 2026.
2. The BTC returns are multiplied by 100 so they are in percent before we fit distributions. This keeps the values away from tiny numbers that can trip up the optimizer. Multiplying by a constant shifts every candidate's log likelihood by the same amount, so the AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) ranking does not change.

3. We compare the Normal, Laplace, Student t, Logistic, and Cauchy distributions. All of them can take negative values and are symmetric around a center, which suits returns. The normal is the baseline and the others have heavier tails, which we expect from crypto jumps.
4. We left out discrete distributions (Geometric, Binomial, Hypergeometric, Poisson) because returns are continuous. We also left out Exponential, Pareto, Weibull, and Log-normal because they only allow positive values, and many returns are negative.
5. Prices move in ticks, so some 1 minute returns are exactly zero. We report that share and still treat the returns as continuous.

1.4 Summary

```
mean_btc = r_btc.mean()
std_btc = r_btc.std(ddof=1)  # Sample
skew_btc = stats.skew(r_btc)
kurt_btc = stats.kurtosis(r_btc)  # excess kurtosis
min_btc = r_btc.min()
max_btc = r_btc.max()
zero_share_btc = (r_btc == 0).mean()

summary_btc = pd.Series({
    "mean (%)": mean_btc,
    "std dev (%)": std_btc,
    "skewness": skew_btc,
    "excess kurtosis": kurt_btc,
    "min (%)": min_btc,
    "max (%)": max_btc,
    "share of exact zeros": zero_share_btc,
})
summary_btc.to_frame("value")
```

	value
mean (%)	0.000117
std dev (%)	0.177022
skewness	5.169890
excess kurtosis	6783.333709
min (%)	-61.705776
max (%)	61.691387
share of exact zeros	0.086240

For a normal distribution the skewness is 0 and the excess kurtosis is 0, so large values of either already hint that normal will not be the winner.

1.5 Methods

We want to know which probability distribution describes the 1-minute BTC log returns best. We compare five candidates: Normal (the baseline), Cauchy, Laplace, Logistic, and Student t. For each distribution we do the following:

1. Estimate the parameters by maximum likelihood with SciPy's `fit`. This finds the parameters that make the observed returns most likely under that distribution.
2. Compute the log likelihood $\ell = \sum_{i=1}^{n} \ln f(x_i)$ at those parameters.
3. Compute AIC $= 2k - 2\ell$ and BIC $= k \ln(n) - 2\ell$, where k is the number of parameters. Lower is better, and the k term stops a distribution from winning just because it has more parameters.

The best fit is the distribution with the lowest AIC, and we check that BIC picks the same one. Fitting five distributions on millions of returns is slow, so we fit on a simple random sample of 100,000 returns without replacement, with a fixed seed so the result reproduces.

1.5.1 Fitting

We define our candidate distributions dictionary and implement the `fit_table` function. This function loops through each distribution, estimates its parameters using maximum likelihood estimation (`dist.fit`), computes the log-likelihood sum, and calculates the AIC and BIC metrics to rank model performance.

```
candidates = {
    "Normal": stats.norm,
    "Cauchy": stats.cauchy,
    "Laplace": stats.laplace,
    "Logistic": stats.logistic,
    "Student t": stats.t,
}

def fit_table(sample_data):
    rows = []
    fits = {}
    for name, dist in candidates.items():
        params = dist.fit(sample_data)
        ll = dist.logpdf(sample_data, *params).sum()
        num_param = len(params)
```

```
        rows.append({
            "distribution": name,
            "k": num_param,
            "log likelihood": ll,
            "AIC": 2 * num_param - 2 * ll,
            "BIC": num_param * np.log(len(sample_data)) - 2 * ll,
        })
        fits[name] = params
    table = pd.DataFrame(rows).sort_values("AIC").reset_index(drop=True)
    table["delta AIC"] = table["AIC"] - table["AIC"].min()
    return table, fits

N_FIT = 100000
rng_btc = np.random.default_rng(SEED)

x_fit = rng_btc.choice(r_btc, size=N_FIT, replace=False)
table_all, fits_all = fit_table(x_fit)
table_all
```

	distribution	k	log likelihood	AIC	BIC	delta AIC
0	Student t	3	93737.231071	-187468.462143	-187439.923366	0.000000
1	Cauchy	2	93358.911059	-186713.822118	-186694.796267	754.640025
2	Laplace	2	85319.048766	-170634.097532	-170615.071681	16834.364611
3	Logistic	2	71158.962611	-142313.925222	-142294.899371	45154.536920
4	Normal	2	38690.318145	-77376.636290	-77357.610439	110091.825853

1.5.2 Repeated Fitting Process

To test whether our distribution ranking is sensitive to sample selection, we run a repeated fitting loop across 10 independent random draws of size $N = 100,000$. We track how many times each candidate achieves the lowest AIC score to confirm the consistency of the winning distribution.

```
reps_btc = 10
wins = pd.Series(0, index=list(candidates))

for i in range(reps_btc):
    x_i = rng_btc.choice(r_btc, size=N_FIT, replace=False)
    table_i, _ = fit_table(x_i)
    wins[table_i.loc[0, "distribution"]] += 1
```

```
wins.to_frame("samples won (lowest AIC)")
```

	samples won (lowest AIC)
Normal	0
Cauchy	0
Laplace	0
Logistic	0
Student t	10

1.5.3 Impact of Zero Returns

Because high-frequency 1-minute crypto data contains periods of low liquidity where prices do not move, exact zero returns can be frequent. To ensure zeros are not distorting our parameter estimates, we filter out all zero-return values and re-run the fitting table.

```
x_nz = x_fit[x_fit != 0]
table_nz, fits_nz = fit_table(x_nz)
print(f"Here are the returns kept without zeros: {len(x_nz):,} of {len(x_fit):,}")
table_nz
```

```
Here are the returns kept without zeros: 91,421 of 100,000
```

	distribution	k	log likelihood	AIC	BIC	delta AIC
0	Student t	3	76264.373067	-152522.746135	-152494.476443	0.000000
1	Cauchy	2	74963.940804	-149923.881608	-149905.035147	2598.864527
2	Laplace	2	69799.935455	-139595.870910	-139577.024449	12926.875224
3	Logistic	2	59309.464845	-118614.929690	-118596.083229	33907.816445
4	Normal	2	31271.098270	-62538.196540	-62519.350079	89984.549595

1.6 Graph

```
best_name = table_all.loc[0, "distribution"]

lim = np.quantile(np.abs(x_fit), 0.999)
bins = np.linspace(-lim, lim, 121)
```

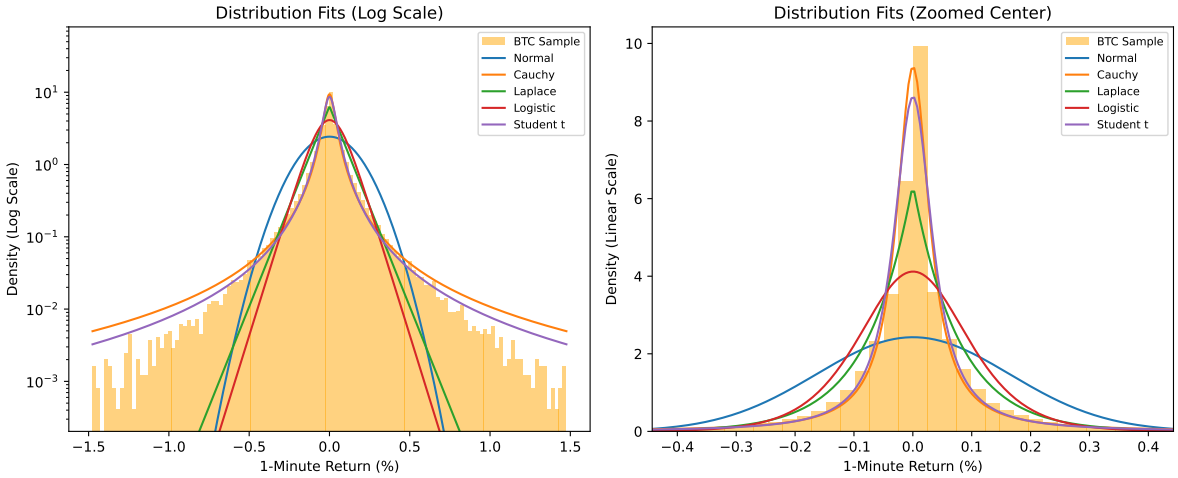
```
grid = np.linspace(-lim, lim, 600)
y_min = 0.5 / (len(x_fit) * (bins[1] - bins[0]))

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Panel 1: Log Scale (Highlights tail fit)
axes[0].hist(x_fit, bins=bins, density=True, alpha=0.5, label="BTC Sample", color="orange")
for name, dist in candidates.items():
    axes[0].plot(grid, dist.pdf(grid, *fits_all[name]), lw=1.5, label=name)
axes[0].set_yscale("log")
axes[0].set_ylim(y_min, None)
axes[0].set_xlabel("1-Minute Return (%)")
axes[0].set_ylabel("Density (Log Scale)")
axes[0].set_title("Distribution Fits (Log Scale)")
axes[0].legend(fontsize=8)

# Panel 2: Linear Scale Zoomed (Highlights peak/center fit)
axes[1].hist(x_fit, bins=bins, density=True, alpha=0.5, label="BTC Sample", color="orange")
for name, dist in candidates.items():
    axes[1].plot(grid, dist.pdf(grid, *fits_all[name]), lw=1.5, label=name)
axes[1].set_xlim(-lim * 0.3, lim * 0.3)
axes[1].set_xlabel("1-Minute Return (%)")
axes[1].set_ylabel("Density (Linear Scale)")
axes[1].set_title("Distribution Fits (Zoomed Center)")
axes[1].legend(fontsize=8)

plt.tight_layout()
plt.show()
```



1.7 Interpretation

Of the five candidates, the best fitting distribution for the 1 minute BTC returns is the Student t distribution. It has the lowest AIC at -187,468, and the runner up, Cauchy, is 755 AIC points behind. The Normal is 110,092 points behind the best. BIC picks Student t, and the fitted parameters of the winner (in the order SciPy returns them) are 1.2167, 0.0003, 0.0382. In the 10 repeated samples, Student t had the lowest AIC in 10 of them, so the result is not the whim of one sample.

Now on to the histogram, the Normal distribution misses the sharp peak and extreme tails, while the Student t tightly matches the empirical data. When the zero returns are removed, the best fit becomes Student t.

2. Shapiro Wilk Test for ETH/USD (5-Minute Returns)

2.1 Question

Test using Shapiro-Wilk normality test the Ethereum returns for trading data every five minutes, from August 7, 2015 to September 15, 2026.

2.2 Data

This part uses two Ethereum datasets. The first is the Kraken trade data (`XETHZUSD.csv`), which lists individual trades starting August 7, 2015. The second is the ETH/USD 1 minute data (`ethusd_1min_ohlc.csv`). Both got resampled to 5 minute bars and then concatenated

at a merging point (August 16, 2017 at 16:45 UTC), so every time period only comes from one source. The 5 minute log returns are calculated after that.

Dataset Links

- **Kraken XETH/USD Trades:** Covers August 7, 2015 to July 4, 2018 featuring tick-by-tick individual transaction logs. https://www.kaggle.com/datasets/pvertenten/kraken-xethusd-trades
- **Ethereum Historical Data:** Covers August 2017 to Present featuring daily updated 1-minute OHLCV intervals. https://www.kaggle.com/datasets/viniciusqroz/ethereum-historical-data

We use `resample_data.py` to merge them, and the results are saved in `data/processed`. Now let's load it.

Now let's load the results.

```
eth_path = ROOT / "data" / "processed" / "eth_5min_returns_2015_2026.csv"
eth = pd.read_csv(eth_path, index_col=0, parse_dates=True)
r = eth["log_return"].to_numpy()

assert np.isfinite(r).all(), "found NaN or inf in the returns"

print(f"Observations: {len(r):,}")
print(f"First Bar: {eth.index.min()}")
print(f"Last Bar: {eth.index.max()}")
```

```
Observations: 999,488
First Bar: 2015-08-08 17:35:00
Last Bar: 2026-09-15 23:55:00
```

2.3 Assumptions

On top of the shared assumptions above:

1. The ETH data comes from two sources. The Kraken trades file (XETHZUSD.csv) runs from August 7, 2015 to July 4, 2018. The 1 minute OHLC file (ethusd_1min_ohlc.csv) runs from about August 2017 to September 29, 2026.
2. Both sources are turned into 5 minute bars, and the close of a bar is the last price (or the last 1 minute close) inside that 5 minute window.
3. The two sources overlap for about 11 months. We use the Kraken bars before August 16, 2017 at 16:45 UTC and the 1 minute bars from then on, so no bar comes from both. The Kraken trades after that point (up to July 2018) are not used.

4. The return that crosses that point is dropped because it compares two different sources.
5. The early Kraken trades are sparse, so many 5 minute bars in the early years have no trades. We put the series on a full 5 minute grid so empty bars become NaN.

2.4 Summary

```
mean_r = r.mean()
std_r = r.std(ddof=1)  # Sample
skewness = stats.skew(r)
excess_kurtosis = stats.kurtosis(r)
min_r = r.min()
max_r = r.max()
zero_share = (r == 0).mean()

summary = pd.Series({
    "mean": mean_r,
    "std dev": std_r,
    "skewness": skewness,
    "excess kurtosis": excess_kurtosis,
    "min": min_r,
    "max": max_r,
    "share of exact zeros": zero_share,
})
summary.to_frame("value")
```

	value
mean	0.000006
std dev	0.004608
skewness	-69.857799
excess kurtosis	32752.610158
min	-1.897120
max	0.818310
share of exact zeros	0.021374

For a normal distribution the skewness is 0 and the excess kurtosis is 0, so large values of either already hint at a problem.

2.5 Methods

The Shapiro-Wilk normality test is a hypothesis test that requires us to form a null and an alternative hypothesis. Our chosen significance level (α) is 0.05, defined as:

- **Null Hypothesis (H_0):** The data is normally distributed.
- **Alternative Hypothesis (H_1):** The data is not normally distributed.

SciPy's `shapiro` function is only reliable up to 5,000 observations (it warns us if our data is above that), so we draw a simple random sample of 5000 returns without replacement, with a fixed seed so our results are reproducible. Then, we repeat the draw 1000 times to make sure one lucky or unlucky sample is not driving the conclusion.

2.5.1 Test on One Sample

```
rng = np.random.default_rng(SEED)
n = 5000

sample = rng.choice(r, size=n, replace=False)
W, p = stats.shapiro(sample)

print(f"Sample Size: {n}")
print(f"Shapiro-Wilk Test Statistic (W): {W:.4f}")
print(f"P-Value: {p:.3e}")
if p < 0.05:
    print("Reject the null hypothesis at the 5% significance level.\n")
    print("Conclusion: The data is not normally distributed.")
else:
    print("Fail to reject the null hypothesis at the 5% significance level.\n")
    print("Conclusion: The data is normally distributed.")
```

```
Sample Size: 5000
Shapiro-Wilk Test Statistic (W): 0.5788
P-Value: 1.754e-76
Reject the null hypothesis at the 5% significance level.

Conclusion: The data is not normally distributed.
```

2.5.2 Repeating the Test 1,000 Times

```
reps = 1000

W_all = np.zeros(reps)
p_all = np.zeros(reps)

for i in range(reps):
  draw = rng.choice(r, size=n, replace=False)
  test_stat, p_value = stats.shapiro(draw)
  W_all[i] = test_stat
  p_all[i] = p_value

median_W = np.median(W_all)
min_W = W_all.min()
max_W = W_all.max()

rejection_rate = (p_all < 0.05).mean()

print(f"Median Test Statistic (W): {median_W:.4f}")
print(f"Minimum Test Statistic (W): {min_W:.4f}")
print(f"Maximum Test Statistic (W): {max_W:.4f}")
print(f"Percentage of Samples Rejecting Normality: {rejection_rate:.4%}")
print(f"Total Number of Random Samples Evaluated: {reps}")
print("Significance Level applied: 5.0000%")
```

```
Median Test Statistic (W): 0.6999
Minimum Test Statistic (W): 0.0267
Maximum Test Statistic (W): 0.8283
Percentage of Samples Rejecting Normality: 100.0000%
Total Number of Random Samples Evaluated: 1000
Significance Level applied: 5.0000%
```

2.6 Graphs

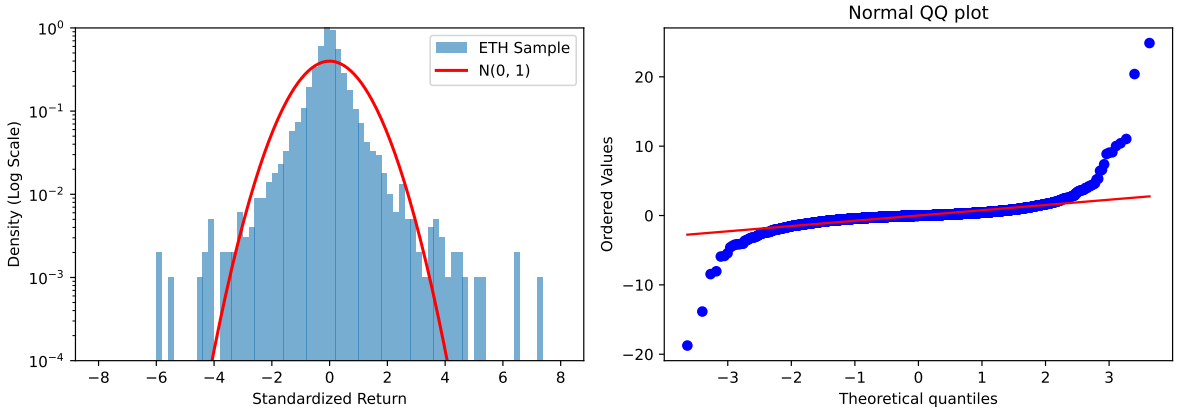
```
z = (sample - sample.mean()) / sample.std(ddof=1)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

```
# Histogram
bins = np.linspace(-8, 8, 81)
axes[0].hist(z, bins=bins, density=True, alpha=0.6, label="ETH Sample")
grid = np.linspace(-8, 8, 400)
axes[0].plot(grid, stats.norm.pdf(grid), "r", lw=2, label="N(0, 1)")
axes[0].set_yscale("log")
axes[0].set_ylim(1e-4, 1)
axes[0].set_xlabel("Standardized Return")
axes[0].set_ylabel("Density (Log Scale)")
axes[0].legend()

# QQ Plot
stats.probplot(z, dist="norm", plot=axes[1])
axes[1].set_title("Normal QQ plot")

plt.tight_layout()
plt.show()
```



2.7 Extremes

```
eth.loc[eth["log_return"].abs().nlargest(10).index, ["close", "volume", "log_return"]]
```

timestamp	close	volume	log_return
2015-08-08 17:40:00	0.15000	1303.031000	-1.897120
2017-05-07 22:50:00	32.00000	6732.191271	-0.916291

timestamp	close	volume	log_return
2017-05-07 23:00:00	68.00000	1438.748259	0.818310
2017-05-07 23:35:00	81.92910	15503.326808	0.742293
2017-05-07 23:25:00	42.00000	9265.960393	-0.510825
2017-05-07 22:35:00	53.10000	12408.311325	-0.496257
2017-05-07 23:20:00	69.99999	8400.908433	0.487295
2017-05-07 23:10:00	44.19389	3975.438715	-0.459909
2017-05-07 22:40:00	82.00000	7064.457804	0.434542
2015-08-16 04:40:00	1.00000	278.000000	-0.336472

2.8 Interpretation

The Shapiro Wilk test on a random sample of 5000 ETH 5 minute log returns gave $W = 0.5788$ and a p value of 1.75e-76. Since the p value is below $\alpha = 0.05$, we reject H_0 and conclude the returns are not normally distributed. The repeated test agrees, with H0 rejected in 100.00% of 1000 samples, so this is not a fluke of one sample.

The full series has an excess kurtosis of 32752.6102 and a skewness of -69.8578. A normal distribution has 0 for both. A kurtosis this large means a handful of extreme returns dominate the series, and the negative skewness means the biggest of them are drops. On the QQ plot the points sit flatter than the line in the middle, since most returns are tiny compared with the overall spread, and then shoot away from the line at both ends. The log scale histogram shows the same pattern, with a taller peak than the red curve and bars out in the tails where the normal density is essentially zero.

A word of caution: With a sample of 5,000, the Shapiro-Wilk normality test is powerful enough to reject for even small departures from normality, so for our purposes rejection alone is not very surprising. The kurtosis and the plots show the departure is large, not just detectable.